BÀI TẬP THỰC HÀNH TỔNG HỢP (120 PHÚT)

XÂY DỰNG RESTFUL API CHUẨN DOANH NGHIỆP THEO QUY TRÌNH API-FIRST VÀ SPRING BOOT 3.X

Chủ đề: Chuẩn hóa RESTful API, OpenAPI 3.0 Specification, Chạy Mockup UI/UX và Tối ưu hóa Backend với JPA & @Transactional

Thời lượng: 120 phút (hoặc Bài tập lớn tuần)	**Đối tượng:** Sinh viên Backend Java / Spring Boot
Công nghệ: Java 21, Spring Boot 3.4+, Spring Data JPA, OpenAPI 3.0, MySQL	**Công cụ:** Swagger Editor, Postman, Maven, IDE (IntelliJ/Eclipse)

I. MỤC TIÊU BÀI TẬP

• **1. Nắm vững nguyên tắc RESTful API chuẩn mực:** Đặt tên tài nguyên đúng quy cách (danh từ số nhiều), áp dụng chuẩn xác phương thức HTTP Verbs, chuẩn hóa mã trạng thái HTTP (Status Codes) và cấu trúc phản hồi JSON.

• **2. Thực hành thành thạo quy trình API-First Development:** Thiết kế hợp đồng API (API Contract) dạng YAML bằng kịch bản OpenAPI 3.0 trước khi triển khai mã nguồn Backend.

• **3. Chạy Mockup API & Phát triển song song:** Khởi chạy máy chủ giả lập (Mock Server) cho phép đội ngũ Frontend / UI-UX thử nghiệm và hoàn thiện giao diện người dùng trước khi có API thật.

• **4. Sinh mã tự động & Tối ưu hóa Backend:** Sử dụng Maven Plugin để sinh tự động Interfaces & DTOs, áp dụng annotation @Transactional(readOnly = true) để tối ưu hóa hiệu năng truy vấn JPA/Hibernate.

II. MÔ TẢ TỔNG QUAN HỆ THỐNG (SHOP API)

Sinh viên tiến hành xây dựng hệ thống **Shop API** quản lý hoạt động kinh doanh trực tuyến bao gồm 3 module tài nguyên chính:

• **Khách hàng (/customers):** Quản lý hồ sơ thông tin cá nhân, địa chỉ và lịch sử tài khoản.

• **Sản phẩm (/products):** Quản lý danh mục, giá bán, mô tả và số lượng tồn kho sản phẩm.

• **Đơn hàng (/orders):** Xử lý đặt hàng, tính toán tổng tiền và cập nhật trạng thái đơn hàng.

III. YÊU CẦU CỤ THỂ DÀNH CHO SINH VIÊN

1. Thiết kế API tuân thủ chuẩn RESTful API

• **Cấu trúc URL (Endpoint Naming):** Sử dụng danh từ số nhiều (plural nouns) đại diện cho tài nguyên, *tuyệt đối không chứa động từ* trên URL (ví dụ: dùng POST /api/v1/orders thay vì /createOrder). Định dạng URL dạng kebab-case và thể hiện quan hệ lồng nhau (nesting) không quá 2 cấp (ví dụ: GET /api/v1/customers/{id}/orders).

• **Sử dụng ngữ nghĩa HTTP Verbs chuẩn:** GET (truy vấn), POST (tạo mới tài nguyên, trả về HTTP 201 Created kèm Header Location), PUT / PATCH (cập nhật toàn bộ hoặc một phần), DELETE (xóa tài nguyên, trả về 204 No Content).

• **Chuẩn hóa Phản hồi JSON & Mã trạng thái HTTP:** Trả về đúng HTTP Status Codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request (lỗi Validation), 404 Not Found, 422 Unprocessable Entity. Cấu trúc JSON báo lỗi thống nhất gồm code, message và details.

• **Hỗ trợ Phân trang (Pagination):** Áp dụng phân trang cho danh sách tài nguyên thông qua Query Parameters (ví dụ: ?page=0&size=10&sort=price,desc).

2. Quy trình API-First Development & Lập trình Backend

Bước 2.1: Thiết kế API Contract (OpenAPI 3.0 Specification)

• Tạo tệp đặc tả src/main/resources/shop-api.yaml tuân theo chuẩn OpenAPI 3.0.3.

• Khai báo đầy đủ thông tin info, servers (http://localhost:8080/api/v1), paths, dữ liệu mẫu (request/response schemas) và các ràng buộc Validation (@NotNull, @Email, @Size, @Min).

Bước 2.2: Chạy Mockup API & Tích hợp với UI/UX Prototype (Fail-Fast)

• Import tệp YAML vào Swagger Editor, Postman Mock Server hoặc Prism để khởi chạy Mock Server trả về dữ liệu mẫu mà chưa cần viết mã Java Backend.

• Kết nối giao diện UI/UX (Frontend/Mobile Prototype) với Mock Server để kiểm thử luồng trải nghiệm người dùng.

• Rà soát và tinh chỉnh bản thảo API ngay nếu phát hiện thiếu trường dữ liệu hoặc cấu trúc phản hồi chưa tối ưu (tiếp cận Fail-Fast).

Bước 2.3: Sinh mã tự động & Lập trình Backend với Spring Boot 3.x

• Cấu hình openapi-generator-maven-plugin trong pom.xml để sinh tự động Interfaces & DTOs/Records vào target/generated-sources.

• Viết các RestController kế thừa Interfaces sinh ra (ví dụ: public class CustomersController implements CustomersApi).

• Tối ưu hóa JPA & Giao dịch: Áp dụng @Transactional(readOnly = true) tại tầng Service cho các thao tác truy vấn đọc để Hibernate bỏ qua cơ chế Dirty Checking, tiết kiệm CPU và RAM.

IV. SẢN PHẨM PHẢI NỘP VÀ TIÊU CHÍ ĐÁNH GIÁ

Hồ sơ sản phẩm sinh viên nộp bao gồm:

• File hợp đồng API Contract: shop-api.yaml (Đúng chuẩn OpenAPI 3.0).

• Báo cáo minh họa / Video thử nghiệm chạy Mockup API kết nối giao diện UI/UX.

• Mã nguồn dự án Spring Boot 3.x hoàn chỉnh (chạy thành công với lệnh mvn spring-boot:run).

• Ảnh chụp kiểm thử thành công các endpoints trên Swagger UI (http://localhost:8080/swagger-ui.html).

Bảng Thang Điểm Đánh Giá Bài Tập (Thang điểm 10):

Tiêu chí đánh giá	Nội dung yêu cầu chi tiết	Trọng số
1. Chuẩn RESTful API	Đặt tên URL đúng quy cách (danh từ số nhiều, kebab-case), sử dụng chuẩn xác HTTP Verbs, mã HTTP Status Codes và cấu trúc JSON lỗi thống nhất.	**3.0 điểm**
2. Quy trình API-First & Mockup API	Xây dựng file OpenAPI 3.0 chuẩn xác, khởi chạy Mock Server thành công, kết nối kiểm thử với giao diện UI/UX tốt theo tinh thần Fail-Fast.	**2.5 điểm**
3. Sinh mã tự động (Codegen)	Cấu hình Maven Plugin chính xác, sinh tự động Interfaces & DTOs vào target directory, Controller kế thừa đúng Interface.	**2.5 điểm**
4. Backend & Tối ưu JPA	Thực thi đúng logic nghiệp vụ, kết nối CSDL MySQL, áp dụng @Transactional(readOnly = true) tối ưu thao tác đọc.	**2.0 điểm**